

Foundations of Artificial Intelligence

M. Helmert
C. Büchner

University of Basel
Spring Term 2026

Exercise Sheet 4 — Solutions

Exercise 4.1 (3.5+1.5+0.5+0.5 marks)

The task in this exercise is to write a Java program. We expect you to implement your code on your own without using existing code or automatically generated code. If you encounter technical problems or have other difficulties with the task, please contact your tutor as early as possible.

The objective of Exercise 2.4 was to implement the state space of the pancake problem. On ADAM, you'll find Java code associated with this exercise sheet (`iterative-deepening.zip`) that implements an interface for search algorithms in the base class `SearchAlgorithmBase`. It builds on the model solution of Exercise 2.4, but you can replace the state space with your own solution if you prefer. For this exercise, you only have to create a new file `IterativeDeepeningSearch.java`.

- (a) Implement iterative deepening search in a new file `IterativeDeepeningSearch.java`. Your class must inherit from `SearchAlgorithmBase` and implement the method `run()`. Print the number of generated nodes (i.e., the number of recursive calls to depth-limited search) for each depth level as well as the total number of generated nodes.

- (b) Test your implementation on the problem instances you can find on ADAM with this exercise sheet. Set a CPU time limit of 10 minutes and a memory limit of 2 GB for each run.

To set a CPU time limit on Linux, use the command `ulimit -St 600`. This will set a soft limit of 600 seconds, after which your Java process will receive a signal to terminate. You can recognize that the task was terminated by the signal by inspecting its exit code. The signal for a time limit is `SIGXCPU`, which will be reported as an exit code of 152. Note that we are limiting CPU time, not wall clock time. Your actual run could be much faster (if Java internally uses multiple cores) or even slower (if other processes are running on your computer). We prefer measuring CPU time over wall clock time, as it gives more reproducible results.

To set the memory limit, run your instances with the parameter `-Xmx2048M`. For example, the first instance can be run like this:

```
java -Xmx2048M IterativeDeepeningSearch pancakes instances/pancakes_prob_01
```

Report runtime, number of generated nodes in the last iteration, total number of generated nodes, and solution length for all instances that can be solved within the given time and memory limits. For all other instances, report whether the time or the memory limit was violated in the column “result”, and report the values of the last statistics output (generated nodes in last iteration and in total and lower bound on solution length) with an added “>”.

Example:

instance	result	runtime	generated nodes		solution length
			last iteration	total	
01	solved	0.02	5 429	7 294	5
02	timeout		> 19 608	> 22 875	> 5
...					

- (c) Problems 01 and 03 have the same solution lengths and therefore find solutions in the same iteration of iterative deepening search. Yet, they generate significantly different numbers of nodes before finding that solution. How is this possible given that both solve instances of the pancake problem?

Hint: What is the main difference between the two problem instances and how does this influence the induced state spaces?

- (d) Does your experiment from part (b) confirm the theoretical claim that the total number of generated nodes is dominated by the last iteration? Justify your answer.

Solution:

- (a) An example implementation of `IterativeDeepeningSearch.java` is available on ADAM.
- (b) An experiment performed with a 10 minute timeout and 2 GB memory limit yielded the following results (OOM means “out of memory”):

instance	result	runtime	generated nodes		solution length
			last iteration	total	
01	solved	0.02	5 429	7 294	5
02	solved	0.04	27 188	50 063	6
03	solved	0.03	34 722	40 071	5
04	solved	0.09	320 633	395 366	6
05	solved	30.21	353 307 323	476 764 112	9
06	solved	112.41	1 325 430 609	1 584 804 854	9
07	timeout		> 5 628 851 293	> 6 140 565 046	> 9
08	timeout		> 883 708 281	> 957 350 637	> 8
09	timeout		> 1 589 311 291	> 1 711 566 005	> 8
10	timeout		> 2 745 954 241	> 2 942 093 829	> 8

- (c) The two problems differ in the number of pancakes: instance 01 has 5 pancakes whereas instance 03 has 7 pancakes. As a consequence, there is a different number of possible choices how many pancakes to flip (i.e., actions) depending on the problem instance. Problem instance 01 therefore has a smaller branching factor than 03, generating fewer states for every considered search depth.
- (d) Yes it does. The number of generated nodes in the last iteration is always at least 50% of the states generated in total, but usually between 75–85% for problems solved within the time and memory limit. In all cases where the algorithm runs out of time (problem instances 07–10), the percentage of generated nodes in the last iteration can be considered worst-case behavior for the considered depth (i.e., a solution is found at the state considered last within that depth bound). For these cases, percentages go beyond 90% up to 93%, increasing with problem size, and rendering previous iterations more and more negligible.

Exercise 4.2 (2 marks)

Consider an instance of the pancakes problem with n pancakes where a state $s = \langle p_1, \dots, p_n \rangle$ represents the sizes of the pancakes from top to bottom. Now consider the following heuristic:

$$h(s) = \sum_{i=1}^{n-1} (|p_i - p_{i+1}| - 1)$$

Intuitively, for each pair of adjacent pancakes p_i, p_{i+1} , we count how many pancakes we still need to put in between them to have all pancakes in order.

Which of the four properties *safe*, *goal-aware*, *admissible* and *consistent* does h satisfy? Justify your answer for each property.

Solution:

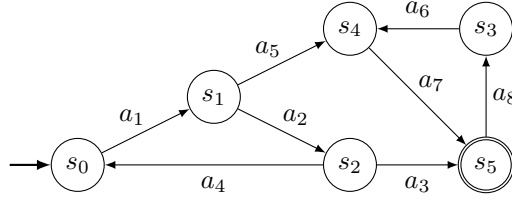
The heuristic is safe and goal-aware but neither admissible nor consistent:

- safe: $h(s)$ is the sum of $n - 1$ numbers from the set $\{0, \dots, n - 2\}$ and therefore finite for all s .
- goal-aware: The only goal state is $s_g = \langle 1, \dots, n \rangle$. For each $1 \leq i \leq n - 1$ we have $|p_i - (i + 1)| = 1$ and thus $h(s_g) = \sum_{i=1}^{n-1} (|p_i - p_{i+1}| - 1) = 0$, i.e., the only goal state has a heuristic value of 0 and thus h is goal-aware.
- inadmissible: Consider the state $s = \langle 3, 2, 1, 4 \rangle$ of a pancake problem with 4 pancakes. While no pancakes need to be put in between p_1, p_2 and p_3 , two pancakes need to be put between $p_3 = 1$ and $p_4 = 4$. We thus have $h(s) = 0 + 0 + 2 = 2$. However, flipping the top three pancakes puts both pancakes of sizes 2 and 3 between them at once. More formally the action sequence $\pi = \langle \text{flip-3} \rangle$ results in the goal state $s_g = \langle 1, 2, 3, 4 \rangle$ and therefore solves state s at cost $\text{cost}(\pi) = 1$, meaning $h^*(s) = 1 < h(s)$.
- inconsistent: Heuristic h cannot be consistent because otherwise it would follow together with goal-awareness that h is also admissible (slide set B10, “Properties of Heuristics: Connections (2)”), which we have shown above to be not the case.

Exercise 4.3 (2 marks)

For the state space depicted below with uniform action costs of 1, define a heuristic that is consistent, not safe and has a perfect estimate for s_0 (i.e., $h(s_0) = h^*(s_0)$). Justify your answer by showing why the heuristic is consistent and not safe.

Hint: If a consistent heuristic assigns ∞ to a state, all its successors must be assigned ∞ as well.



Solution:

The heuristic $h = \{s_0 \mapsto 3, s_1 \mapsto 2, s_2 \mapsto 1, s_3 \mapsto \infty, s_4 \mapsto \infty, s_5 \mapsto \infty\}$ is a possible solution. Since s_5 is a goal state we have $h^*(s_5) = 0$, and since $h(s_5) = \infty$ we can conclude that h is not safe. To verify consistency we need to show that every transition is consistent:

$$\begin{array}{ll}
 \langle s_0, a_1, s_1 \rangle : 3 \leq 1 + 2 = 3 & \langle s_1, a_2, s_2 \rangle : 2 \leq 1 + 1 = 2 \\
 \langle s_2, a_3, s_5 \rangle : 1 \leq 1 + \infty = \infty & \langle s_2, a_4, s_0 \rangle : 1 \leq 1 + 3 = 4 \\
 \langle s_1, a_5, s_4 \rangle : 2 \leq 1 + \infty = \infty & \langle s_3, a_6, s_4 \rangle : \infty \leq 1 + \infty = \infty \\
 \langle s_4, a_7, s_5 \rangle : \infty \leq 1 + \infty = \infty & \langle s_5, a_8, s_3 \rangle : \infty \leq 1 + \infty = \infty
 \end{array}$$

Some observations:

- If with a consistent heuristic h we have $h(s) = \infty$ for some state s , then we have $h(s') = \infty$ for all states reachable from s . Since a unsafe heuristic has $h(s) = \infty$ for some s from which we can reach a goal, then we can conclude that all goal states s_g reachable from s must have $h(s_g) = \infty$. In this example, this means that $h(s_5) = \infty$ must hold since it is the only goal state, and since s_4 and s_3 are reachable from s_5 we also have $h(s_4) = h(s_3) = \infty$.
- If with a consistent heuristic h we have $h(s) < \infty$ for some state s , then all states which reach s must also have $h(s) < \infty$. In this example, this means that $h(s_1) < \infty$ and $h(s_2) < \infty$.